Fiat 600

Medium Cryptography Challenge - Writeup

"What about fiat seicento? We bet you can drive this one as well."

Category: Cryptography

Difficulty: Medium

Type: Interactive (netcat)

1. Background: Schnorr Identification Protocol

The Schnorr identification protocol is a zero-knowledge proof system that allows someone (the Prover) to convince someone else (the Verifier) that they know a secret key, without revealing the secret itself.

It works in a group of prime order Q with a generator g. The Prover has:
- Private key: x (a random secret number)
- Public key: H = g^x mod Q

The protocol has three steps:

1.1 The Three Steps

Step 1 - COMMITMENT: The Prover picks a random number r and computes:
u = g^r mod Q
The Prover sends u to the Verifier.

Step 2 - CHALLENGE: The Verifier sends a random challenge c to the Prover.
(In non-interactive mode, c = Hash(g, Q, H, u))

Step 3 - RESPONSE: The Prover computes:
z = r + x * c (mod Q)
The Prover sends z to the Verifier.

The Verifier checks that: g^z == u * H^c mod Q

This works because: g^z = g^(r+xc) = g^r * g^(xc) = u * (g^x)^c = u * H^c

1.2 Why Is It Secure?

The security relies on the fact that the challenge c MUST depend on the commitment u. If c is computed BEFORE u is known, or if c does not include u in its computation, then the Prover can cheat! This is the critical point exploited in this challenge.

2. Challenge Analysis

2.1 The Server Code

The server implements a Schnorr verification scheme. The key function is SchnorrVerify:

```
def SchnorrVerify(h, u, c, z):
    if u * h * z % Q == 0:
        return False

    tuple_hash = TupleHash128.new(digest_bytes=400)
    tuple_hash.update(GENERATOR.to_bytes(400, 'big'))
    tuple_hash.update(Q.to_bytes(400, 'big'))
    tuple_hash.update(h.to_bytes(400, 'big'))

    c_prime = int.from_bytes(tuple_hash.digest(), 'big')

    hashes_match = c == c_prime
    valid_eq = pow(GENERATOR, z, Q) == u * pow(h, c, Q) % Q

    return hashes_match and valid_eq
```

2.2 The Vulnerability

Look carefully at what goes INTO the hash:

c = Hash(GENERATOR, Q, h)

In a CORRECT Schnorr protocol, the hash should be:

c = Hash(GENERATOR, Q, h, u) <-- 'u' is MISSING!

The commitment 'u' is NOT included in the hash! This means the challenge 'c' is a FIXED, DETERMINISTIC value that depends only on the public parameters (g, Q) and the public key (H). Anyone can compute it without knowing the private key.

This completely breaks the security of the protocol.

2.3 Why Missing 'u' Breaks Everything

In the correct protocol, the Prover must:

1. First commit to u = g^r (choose r randomly)

2. Then receive/compute c = Hash(..., u)
3. Then compute z = r + x*c

The Prover needs to know x (the private key) to compute z in step 3, because r was already fixed in step 1 before c was known.

But if c doesn't depend on u, an attacker can:

1. First compute c (since it's deterministic)
2. Choose any z (e.g., z = 1337)
3. COMPUTE u backwards: u = g^z * H^(-c) mod Q

This satisfies the verification equation g^z == u * H^c because:

u * H^c = g^z * H^(-c) * H^c = g^z (checkmark!)

No private key needed!

3. The Attack

3.1 Step-by-Step Forgery

Given only the public key H (printed by the server), we forge a valid proof:

1. Compute the deterministic challenge:
   c = TupleHash128(GENERATOR || Q || H)

2. Pick any z (we choose z = 1337, any non-zero value works):
   z = 1337

3. Compute u by rearranging the verification equation:
   g^z == u * H^c mod Q
   u = g^z * H^(-c) mod Q
   u = pow(2, 1337, Q) * pow(H, -c, Q) % Q

4. Submit (u, c, z) to the server. It verifies and gives us the flag!

3.2 Complete Exploit Code

```
from pwn import *
from Crypto.Hash import TupleHash128

GENERATOR = 0x2
Q = 4863582...  # (large prime, from source)

r = remote('challenges.cybersecuritychallenge.pt', 24303)
r.recvuntil(b'public key: ')
H = int(r.recvline().strip())

# Compute deterministic c (the bug: no 'u' in hash!)
tuple_hash = TupleHash128.new(digest_bytes=400)
tuple_hash.update(GENERATOR.to_bytes(400, 'big'))
tuple_hash.update(Q.to_bytes(400, 'big'))
tuple_hash.update(H.to_bytes(400, 'big'))
c = int.from_bytes(tuple_hash.digest(), 'big')

# Forge: choose z, compute u backwards
z = 1337
u = (pow(GENERATOR, z, Q) * pow(H, -c, Q)) % Q

# Submit forged proof
r.sendlineafter(b'choice: ', b'1')
r.sendlineafter(b'u: ', str(u).encode())
r.sendlineafter(b'c: ', str(c).encode())
r.sendlineafter(b'z: ', str(z).encode())
print(r.recvall(timeout=5).decode())
```

4. Local Verification

Since the CTF server may not always be available, we can verify the exploit locally by simulating the server. The script below proves the forgery works against ANY randomly generated private key:

```
from Crypto.Util.number import getRandomInteger
from Crypto.Hash import TupleHash128

# Server generates a key pair (we DON'T know x)
x = getRandomInteger(3071)
H = pow(GENERATOR, x, Q)

# Attacker computes c (deterministic, no u!)
th = TupleHash128.new(digest_bytes=400)
th.update(GENERATOR.to_bytes(400, 'big'))
th.update(Q.to_bytes(400, 'big'))
th.update(H.to_bytes(400, 'big'))
c = int.from_bytes(th.digest(), 'big')

# Forge proof
z = 1337
u = (pow(GENERATOR, z, Q) * pow(H, -c, Q)) % Q

# Verify: this PASSES!
assert SchnorrVerify(H, u, c, z) == True
print('Forgery successful!')
```

Output: Forgery successful! (Tested and confirmed.)

5. Correct vs. Broken Protocol

5.1 Side-by-Side Comparison

BROKEN (this challenge):

c = Hash(g, Q, H) <-- u NOT included

Result: c is predictable, proof is forgeable

CORRECT Schnorr:

c = Hash(g, Q, H, u) <-- u IS included

Result: c depends on the commitment, unforgeable

When u is included in the hash, the attacker faces a chicken-and-egg problem:

- To compute u backwards, you need to know c first

- But c depends on u (via the hash)
- Finding u such that Hash(..., u) yields a c that satisfies the equation is computationally infeasible (requires breaking the hash function)

6. Lessons Learned

6.1 The Fiat-Shamir Heuristic Must Include the Commitment

The Fiat-Shamir heuristic converts an interactive proof into a non-interactive one by replacing the verifier's random challenge with a hash. The hash MUST include ALL messages sent before the challenge, especially the commitment (u). Omitting it makes the proof trivially forgeable.

The challenge name 'Fiat 600' is a pun on the Fiat-Shamir transform and the Fiat car brand.

6.2 Understanding the Math

The verification equation is: $g^z = u * H^c \pmod Q$

If the attacker can choose u AFTER knowing c, they can solve for u:
$u = g^z / H^c = g^z * H^{(-c)} \pmod Q$

This is trivial modular arithmetic. The ONLY thing that prevents this in a correct protocol is that c must depend on u, creating a circular dependency that cannot be broken without knowing the discrete log (private key x).

7. Summary

This challenge implemented a flawed Schnorr identification scheme where the non-interactive challenge hash omitted the commitment value 'u'. This allowed us to:

1. Compute the deterministic challenge c from public information
2. Choose an arbitrary response z
3. Compute the commitment u backwards to satisfy the verification equation
4. Submit the forged proof (u, c, z) to get the flag

The fix is simple: include u in the hash computation.

Flag: CSCPT{...} (obtained from remote server)